

Geoscience Data Analysis

D. Hasterok (derrick.hasterok@adelaide.edu.au)

Dept. of Earth Sciences, University of Adelaide, North Terrace, Adelaide, SA 5005

8 Multivariate Analysis

Table of Contents

Geoscience Data Analysis.....	1
8 Multivariate Analysis.....	1
8.1 Visualizing Multivariate Data.....	1
8.1.1 Introduction.....	1
8.1.2 Heat Map.....	2
8.1.2 Scatter Plot.....	3
8.1.3 Contour/Surface Plots.....	3
8.1.4 Radar Plot.....	4
8.2 Basic Statistics.....	5
8.2.1 Mean.....	5
8.2.2 Sum of squares.....	6
8.2.3 Deviation (Centering).....	6
8.2.4 Variance and Covariance.....	6
8.2.5 Correlation.....	7
8.3 Metrics.....	7
8.3.1 Definition.....	7
8.3.2 Euclidian Distance.....	7
8.3.3 Mahalanobis Distance.....	7
8.4 Principal Component Analysis.....	8
8.4.1 Relationship to Eigenvalue Decomposition.....	8
8.4.2 Relationship to SVD Decomposition.....	10
8.5 Multi-dimensional Scaling.....	11
8.5.1 Classical MDS	11
8.5.2 Non-classical MDS.....	13
8.6 Linear Discriminant Analysis.....	15

```
clear all;

addpath codes
addpath data
```

8.1 Visualizing Multivariate Data

8.1.1 Introduction

A lot of our data are multivariate as it is increasingly less common to collect one type of data from a sample set for a study. It is possible to represent multiple dimensions for comparison, think the classic Cartesian plot which allows for 2-dimensional representation of a dataset.

Data of 3-dimensions can be visualized in on a 2-dimensional page as a perspective (or even better as virtual reality using stereo-plotting), as 2-dimensional histograms (heat maps), x-y plot with points of different shapes, sizes or colors, and of course in geochemistry as a ternary diagram.

Data of 4-dimensions can be visualized by combining two of the elements above, e.g., scatter plot with color and size, or shape and color, or shape and size.

But how do we represent 5-dimensions, or 6 or 7 or N ? Geochemical data can have 90 dimensions (H through U minus the two non-natural elements). Enter dimensional reduction techniques. Dimensional reduction methods are any mathematical techniques that seek to reduce the number of dimensions to those that highlight the greatest differences and/or similarities in the data with the fewer dimensions than the original data. Dimensional reduction can be performed by removing largely redundant dimensions, or via transformation of the data.

In this chapter, we discuss two-dimensional reduction techniques, principal component analysis (PCA) and multi-dimensional scaling (MDS). There are some similarities between the two, but the results are sufficiently different. Be aware that there are a number of other dimensional reduction techniques not discussed (discriminant analysis, factor analysis, etc.). Depending upon your problem, it may be worthwhile to explore a few of these techniques to find the one that best suits your problem or data.

But before we get to the multidimensional methods, let's look at some ways to visualize multidimensional data, because it is not always necessary to use dimensional reduction to get something out of a multidimensional dataset. The examples below are by no means the only ones. We will examine a few additional multidimensional plots in the geochemistry chapter that are not covered here.

8.1.2 Heat Map

A heat map is simply a 2-dimensional histogram. So we have a set of bins in an x and y dimension.

```
% load plutonic major element geochemistry data
data = readtable('plutonic_geochem.xlsx');

% make a heat map on a total alkali silica plot
S = data.sio2;
TA = data.na2o + data.k2o;

s_ax = [35:1:90];
ta_ax = [0:0.5:18];

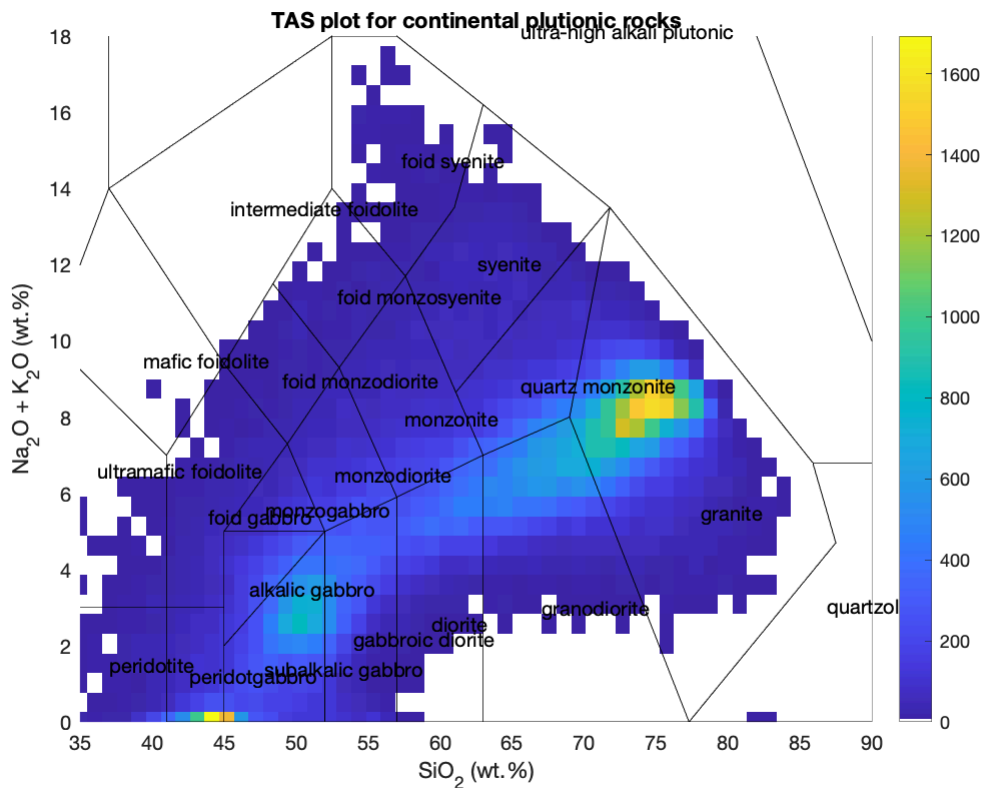
figure;
hold on;
n = hist2d(S,TA,s_ax,ta_ax);
imagesc(s_ax,ta_ax,n);
axis xy;
xlabel('SiO_2 (wt.%)');
ylabel('Na_2O + K_2O (wt.%)');
colormap([1 1 1; parula]);
colorbar;

% add TAS fields
tasgons = load_tasgons;
for i = 1:length(tasgons)
    x = tasgons{i,1}(:,1);
    y = tasgons{i,1}(:,2);
    plot(x,y,'k-');
```

```

text(mean(x),mean(y),tasgons{i,3});
end
axis([s_ax(1) s_ax(end),ta_ax(1),ta_ax(end)])
set(gca,'Box','on');
title('TAS plot for continental plutonic rocks')

```



8.1.2 Scatter Plot

A scatter plot is simply point by point plot of data. The points can change size, shape, and color to add additional dimensions or emphasize certain aspects of a dataset. We have already seen scatter plots used in previous worked examples in this course.

8.1.3 Contour/Surface Plots

The difference between a contour plot and a surface plot and other similar plots is how the data are constructed while the underlying data may be the same. Several methods to produce such plots are shown below.

```

% create a regular spatial grid
[xq,yq] = meshgrid(-3:0.1:3);

% compute values of a periodic function in x and y
v = sin(xq).^4 .* cos(yq);

figure;
subplot(221);
% use contourf to fill the contours much like pcolor and surface, but the
% colors are discretized according to the contour interval. Matlab's
% contouring algorithm is not the best as it will often draw very angular

```

```

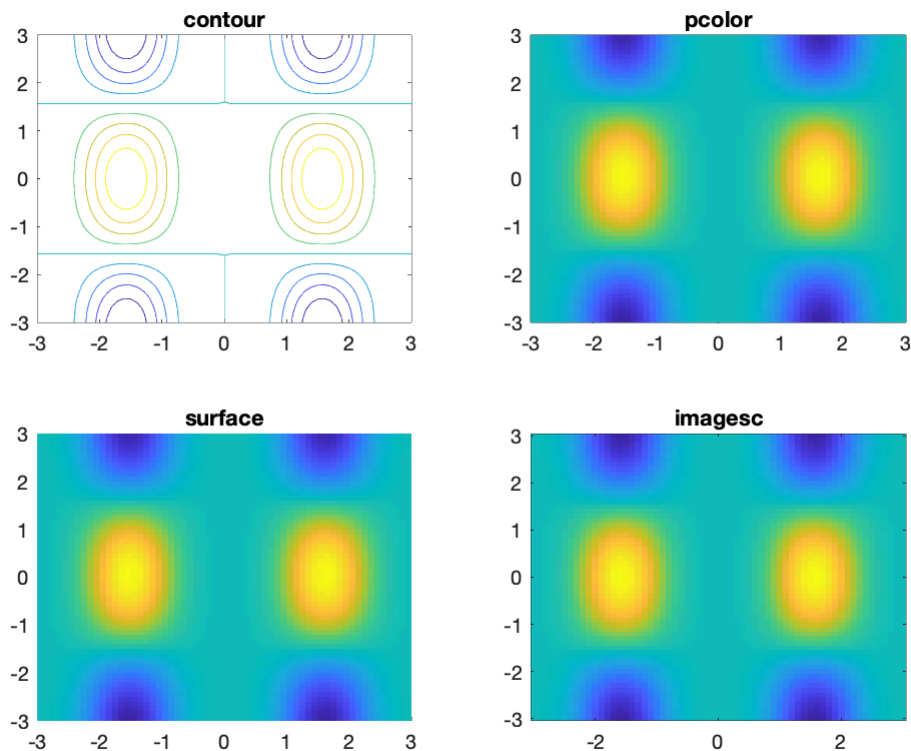
% contours when the gridding is sparse.
contour(xq,yq,v);
title('contour');

subplot(222);
pcolor(xq,yq,v);
title('pcolor');
shading flat;

subplot(223);
surface(xq,yq,v);
title('surface');
shading flat;
axis tight;

subplot(224);
% this method is fastest for the computer, but also the least customizable
% in appearance other than color
imagesc(xq(1,:),yq(:,1),v);
axis xy;
title('imagesc');

```



8.1.4 Radar Plot

A radar plot is a multidimensional plot that produces an axis for each element of a vector extending from the center to the perimeter of an encompassing circle. Each axis is like the spoke of a wheel, separated by equal angles. To plot the data, one then connects the value of each point to the value on the adjacent axes. In the

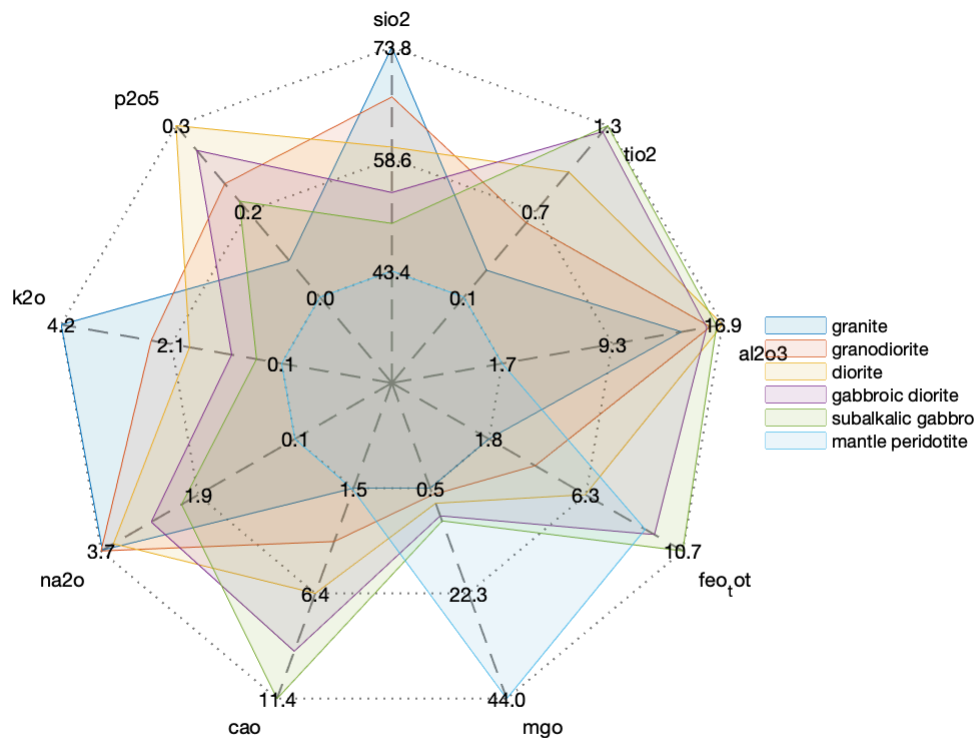
example below, a radar plot is produced to illustrate the changing chemistry of subalkaline plutonic rocks from mafic to felsic compositions. The result nicely illustrates the decrease in CaO, MgO, and FeO as plutonic rocks become more felsic.

```
% using the continental plutonic rocks again as the input dataset

% we will examine the pattern of major elements for several common rock
% types
oxides = {'sio2','tio2','al2o3','feo_tot','mgo','cao','na2o','k2o','p2o5'};
rock_types = {'granite','granodiorite','diorite','gabbroic diorite','subalkalic gabbro'};

% radarprep simply determines the average chemistry for each rock type
R = radarprep(data,'rock_type',rock_types,oxides);

% create radar plot
figure;
radarplot(R);
```



8.2 Basic Statistics

8.2.1 Mean

To compute the mean of a column vector of length M ,

$$\bar{x} = \mathbf{1}^T \mathbf{x} / M$$

and to compute the mean of a matrix by column with size (M, N) ,

$$\bar{\mathbf{x}}^T = \mathbf{1}^T \mathbf{X} / M$$

where $\bar{\mathbf{x}}$ is a column vector.

8.2.2 Sum of squares

Sum of squares. If $\mathbf{x}$ is a column vector,

$$\sum_{i=1}^N x_i^2 = \mathbf{x}^T \mathbf{x}$$

or if $\mathbf{x}$ is a row vector,

$$\sum_{i=1}^N x_i^2 = \mathbf{x} \mathbf{x}^T$$

For a set of multivariate data provided in a matrix, $\mathbf{X}$, with size (M, N) where each row is a sample and each column is a characteristic.

8.2.3 Deviation (Centering)

Centering is simply the process of shifting the data by removing the mean. It places the mean of the centered vector about 0. To center a column vector, $\mathbf{x}_c = \mathbf{x} - \bar{\mathbf{x}} \mathbf{1}$, and to center a matrix, $\mathbf{X}_c = \mathbf{X} - \mathbf{1} \bar{\mathbf{x}}^T$, where each column is a sample and each row a dimension of $\mathbf{X}$ and $\bar{\mathbf{x}} = (\bar{x}_1, \bar{x}_2, \dots, \bar{x}_N)^T$.

8.2.4 Variance and Covariance

For a set of centered multivariate data provided in a matrix, $\mathbf{X}$, with size (M, N) where each row is a sample and each column is a characteristic, i.e., $\mathbf{X} = (\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N)$ where each $\mathbf{x}_j$ is a column vector for an individual variable, each with length M .

$$\Sigma = \text{Cov}(\mathbf{X}, \mathbf{X}) = \frac{\mathbf{X}^T \mathbf{X}}{N-1} = (N-1)^{-1} \begin{pmatrix} \sum x_{1,i}^2 & \sum x_{1,i} x_{2,i} & \sum x_{1,i} x_{3,i} & \cdots & \sum x_{1,i} x_{N,i} \\ \sum x_{2,i} x_{1,i} & \sum x_{2,i}^2 & \sum x_{2,i} x_{3,i} & \cdots & \sum x_{2,i} x_{N,i} \\ \sum x_{3,i} x_{1,i} & \sum x_{3,i} x_{2,i} & \sum x_{3,i}^2 & \cdots & \sum x_{3,i} x_{N,i} \\ \vdots & \vdots & & \ddots & \vdots \\ \sum x_{N,i}^2 & \sum x_{N,i} x_{2,i} & \sum x_{N,i} x_{3,i} & \cdots & \sum x_{N,i}^2 \end{pmatrix}$$

put another way,

$$\Sigma = \begin{pmatrix} \text{Var}(\mathbf{X}_1) & \text{Cov}(\mathbf{X}_1, \mathbf{X}_2) & \text{Cov}(\mathbf{X}_1, \mathbf{X}_3) & \cdots & \text{Cov}(\mathbf{X}_1, \mathbf{X}_N) \\ \text{Cov}(\mathbf{X}_2, \mathbf{X}_1) & \text{Var}(\mathbf{X}_2) & \text{Cov}(\mathbf{X}_2, \mathbf{X}_3) & \cdots & \text{Cov}(\mathbf{X}_2, \mathbf{X}_N) \\ \text{Cov}(\mathbf{X}_3, \mathbf{X}_1) & \text{Cov}(\mathbf{X}_2, \mathbf{X}_3) & \text{Var}(\mathbf{X}_3) & \cdots & \text{Cov}(\mathbf{X}_3, \mathbf{X}_N) \\ \vdots & \vdots & & \ddots & \vdots \\ \text{Cov}(\mathbf{X}_N, \mathbf{X}_1) & \text{Cov}(\mathbf{X}_N, \mathbf{X}_2) & \text{Cov}(\mathbf{X}_N, \mathbf{X}_3) & \cdots & \text{Var}(\mathbf{X}_N) \end{pmatrix}$$

This matrix is also called the variance–covariance matrix. Because $\text{Cov}(\mathbf{X}_a, \mathbf{X}_b) = \text{Cov}(\mathbf{X}_b, \mathbf{X}_a)$, the matrix is symmetric, i.e., $\Sigma^T = \Sigma$.

Note that in Matlab as with correlation functions in other languages handle the centering.

8.2.5 Correlation

Since the diagonals of the covariance matrix are the variances of each variable, we can extract them to produce a new matrix,

$$\mathbf{D} = \text{diag}(\Sigma) = \begin{pmatrix} \sigma_1^2 & 0 & 0 & \dots & 0 \\ 0 & \sigma_2^2 & 0 & \dots & 0 \\ 0 & 0 & \sigma_3^2 & \dots & 0 \\ \vdots & \vdots & & \ddots & \vdots \\ 0 & 0 & 0 & \dots & \sigma_N^2 \end{pmatrix}$$

We can create a correlation matrix by

$$\mathbf{C} = \text{Corr}(\mathbf{X}, \mathbf{X}) = \mathbf{D}^{-1/2} \Sigma \mathbf{D}^{-1/2}$$

Note that in Matlab as with correlation functions in other languages handle the centering.

```
% Compute a covariance matrix for arc properties?
```

8.3 Metrics

8.3.1 Definition

A metric is simply a measure of distance between two points. A metric must satisfy four criteria, for three points $\mathbf{x}_1$, $\mathbf{x}_2$, and $\mathbf{x}_3$

1. distance must be positive or zero, $d(\mathbf{x}_1, \mathbf{x}_2) \geq 0$.
2. a zero distance only occurs where the two points are the same, $d(\mathbf{x}_1, \mathbf{x}_2) = 0 \iff \mathbf{x}_1 = \mathbf{x}_2$.
3. distance is independent of order, $d(\mathbf{x}_1, \mathbf{x}_2) = d(\mathbf{x}_2, \mathbf{x}_1)$.
4. the distance along a multipoint path must be greater than or equal to the distance between the endpoints, $d(\mathbf{x}_1, \mathbf{x}_3) \leq d(\mathbf{x}_1, \mathbf{x}_2) + d(\mathbf{x}_2, \mathbf{x}_3)$.

8.3.2 Euclidian Distance

You are probably most familiar with Euclidian distance as it is how we describe distances in normal Cartesian space. The Euclidian distance is the L_2 norm between two points,

$$d(\mathbf{x}_1, \mathbf{x}_2) = \|\mathbf{x}_1 - \mathbf{x}_2\|_2 = \left[\sum_{i=1}^N (x_{1,i} - x_{2,i})^2 \right]^{1/2}$$

8.3.3 Mahalanobis Distance

The Mahalanobis distance measures how many standard deviations a point lies from a multivariate mean. If all variables of a sample sit at the mean value, the distance is zero. Consider a sample $\mathbf{x}_i$ drawn from a distribution of samples, $\mathbf{X}$, with a mean for each variable, which we can express as a vector $\bar{\mathbf{x}}$, and the covariance matrix for the set of samples, Σ . We can define a distance that is centered about the mean and rescaled by the covariance,

$$D_M(\mathbf{x}_i) = [(\mathbf{x}_i - \bar{\mathbf{x}})^T \Sigma^{-1} (\mathbf{x}_i - \bar{\mathbf{x}})]^{1/2}$$

We can generalize the distances between any two points, $\mathbf{x}_1$ and $\mathbf{x}_2$,

$$d(\mathbf{x}_1, \mathbf{x}_2) = [(\mathbf{x}_1 - \mathbf{x}_2)^T \Sigma^{-1} (\mathbf{x}_1 - \mathbf{x}_2)]^{1/2}$$

8.4 Principal Component Analysis

Principal component analysis (PCA) attempts to define a set of vectors, derived from multivariate data, that maximizes the collective variance of the dataset. With most sample sets, the multivariate characteristics are rarely ever completely independent. In math terms, we would say that the variables are not orthogonal. In a pure system, orthogonality occurs when the correlation coefficient between two parameters is zero.

Consider for example geochemistry of igneous rocks. The process of fractionation from mafic to felsic compositions is responsible for the majority of the compositional variations seen in igneous geochemistry. Mafic igneous rocks by definition have low silica and felsic rocks high silica. Mafic rocks also tend to be high in magnesium, iron, calcium, titanium and manganese whereas mafic rocks tend to be higher in potassium and sodium. Hence, the changes in SiO_2 concentration is not entirely independent of MgO , FeO , etc., and we would expect a combination of these parameters could indicate how mafic or felsic a rock is. But there are other processes and sources that may lead to chemical variations that are independent of fractionation that lead to additional variance. Principal component analysis potentially offers a way in which the variance resulting from these different processes may be separated for study. A note of caution that PCA is a mathematical technique that assumes systems are basically linear in their responses. A PCA will yield a result regardless of the validity of this linearity assumption or the processes involved, so it is up to geoscientist to provide the physical context to interpret the results and ensure that the assumptions are reasonably valid.

Principal component analysis can be related to a few matrix decomposition methods.

8.4.1 Relationship to Eigenvalue Decomposition

Given a matrix $\mathbf{A}$ ($N \times N$), there exist a scalar and vector pair such that

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v},$$

where λ is a scalar (possibly complex) and $\mathbf{v}$ ($N \times 1$) is an associated non-zero vector. The problem may also be phrased as

$$(\mathbf{A} - \lambda\mathbf{I})\mathbf{v} = 0.$$

If $\mathbf{v}$ is non-zero, then the result is only zero when $|\mathbf{A} - \lambda\mathbf{I}| = 0$ (note this calls for the determinant). To find λ , we must solve for the roots to an N -th order polynomial. As a result, there will be N eigenvalues and N associated eigenvectors. You can think of these eigenvalues as the fundamental modes of a system and the eigenvectors

as the shape of the modes. The vectors can be combined into a matrix to create an orthogonal basis for the system, i.e. the dot products of any two eigenvectors is zero, $\mathbf{v}_i \cdot \mathbf{v}_j = 0$ for $i \neq j$.

Thus the full eigenvalue solution can be expressed as

$$\mathbf{A}\mathbf{V} = \mathbf{\Lambda}\mathbf{V},$$

where $\mathbf{V}$ is a matrix of eigenvectors and $\mathbf{\Lambda}$ is a column vector of eigen values.

The principal component analysis of a dataset can be produced by performing an eigenvalue decomposition of the covariance matrix, producing a set of principal components (eigenvectors) with eigenvalues, which relate to the variance explained by each vector. The full set of eigenvalues and vectors can be summarized by

$$\mathbf{\Sigma}\mathbf{V} = \mathbf{\Lambda}\mathbf{V},$$

where $\mathbf{V}$ is the matrix of eigenvectors and $\mathbf{\Lambda}$ is the vector of eigenvalues. We sort the eigenvalues from largest to smallest (by the largest to smallest spread in the data) and arrange the vectors in $\mathbf{V}$ accordingly. The explained variance for each eigenvector is determined by

$$\text{explained variance} = 100 \frac{\lambda_i}{\sum_{j=1}^N \lambda_j}.$$

As you will soon see, only the first few principal components typically account for the vast majority of the variance in the data, suggesting we can disregard the higher principal components with little effect on any data analysis. Therefore, we can create a projection matrix $\mathbf{W}$ from the first k principal components where $k < N$. We can then create transformed dataset,

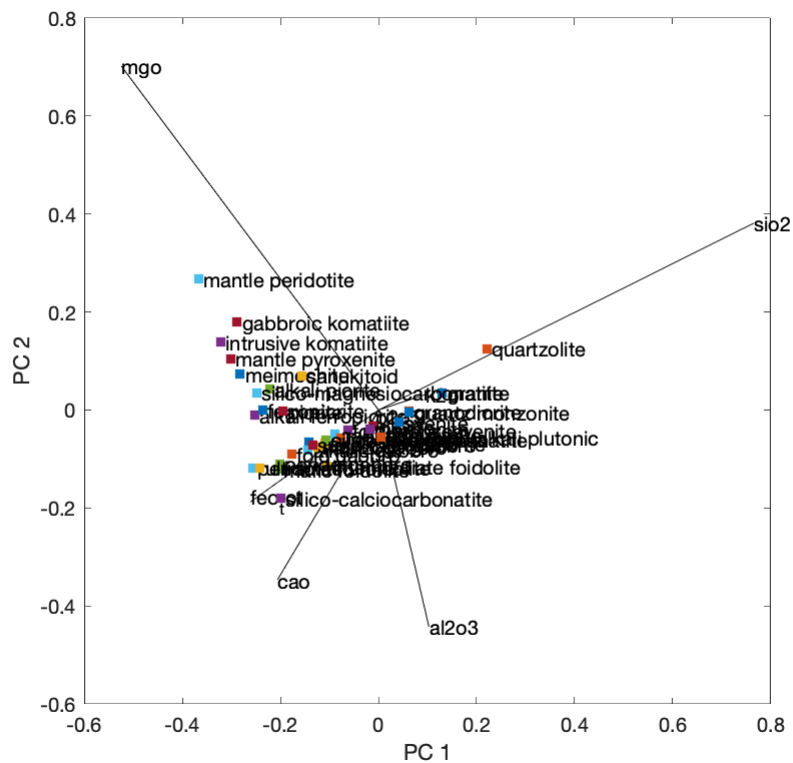
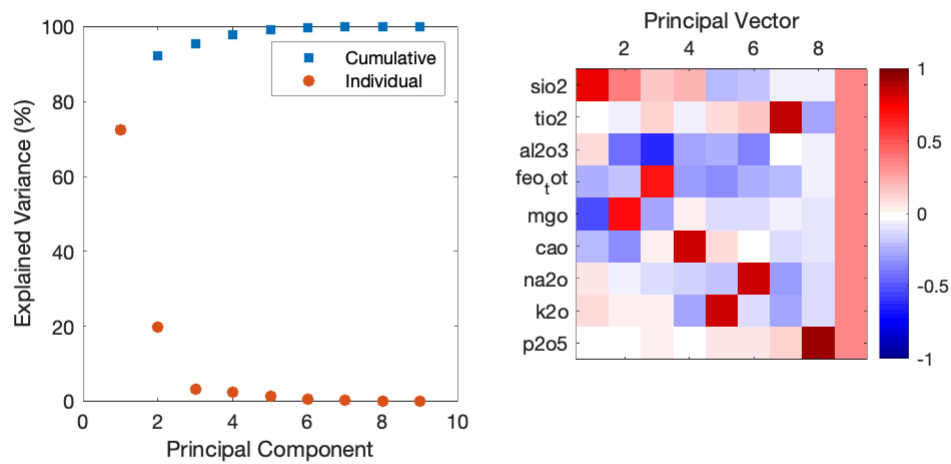
$$\hat{\mathbf{X}} = \mathbf{X}\mathbf{W}$$

which has fewer dimensions than the original, but the same characteristics as the original. This transformation is commonly referred to as dimensionality reduction.

Below is an example of a PCA analysis applied to the major element chemistry of plutonic rocks. We'll revisit this a little further in the geochemical chapter.

```
db_pca(data);
```

```
Warning: Columns of X are linearly dependent to within machine precision.  
Using only the first 8 components to compute TSQUARED.
```



8.4.2 Relationship to SVD Decomposition

Singular value decomposition is a faster way to obtain PCA information. Every matrix can be decomposed into the product of three matrices,

$$\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^T$$

where $\mathbf{U}$ and $\mathbf{V}$ are the unitary matrices which are rotations, $\mathbf{S}$ is a diagonal matrix of singular values that creates a scaling factor. Both $\mathbf{U}$ and $\mathbf{V}$ are constructed from unit vectors such that if both are real-valued, $\mathbf{U}\mathbf{U}^T = \mathbf{I}$ and $\mathbf{V}\mathbf{V}^T = \mathbf{I}$. Using the definition of the covariance matrix, $\mathbf{\Sigma} = \mathbf{X}^T\mathbf{X}(N-1)^{-1}$,

$$\mathbf{\Sigma} = \mathbf{V}\mathbf{S}\mathbf{U}^T\mathbf{U}\mathbf{S}\mathbf{V}^T(N-1)^{-1} = \mathbf{V} \frac{\mathbf{S}^2}{N-1} \mathbf{V}^T$$

Multiplying the right side of each by $\mathbf{V}$, we find

$$\mathbf{\Sigma}\mathbf{V} = \mathbf{V} \frac{\mathbf{S}^2}{N-1}$$

which is the eigenvalue decomposition if $\mathbf{V}$ is the matrix of principle components and $\mathbf{\Lambda} = \frac{\mathbf{S}^2}{N-1}$.

Returning to $\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^T$, we can see that $\hat{\mathbf{X}} = \mathbf{U}\mathbf{S}$ is the transformed dataset.

8.5 Multi-dimensional Scaling

Multidimensional scaling is a method that attempts to quantify the distances between samples as a measure of their similarity. The distance between two samples can be quantified using one of the metrics discussed in [Section 8.3](#). But rather than the true distance, the MDS distance is estimated from dimensionally reduced set of transformed variables—often of two or three coordinates so the result can be visualized. The results are then presented on a Cartesian plot where points that plot close together are more similar in characteristics than points that plot further away.

8.5.1 Classical MDS

The classical MDS is basically PCA. MDS can be reduced to a PCA in some cases as shown by the example below. Consider the SVD decomposition of the centered data matrix $\mathbf{X}$,

$$\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^T$$

If we assume the sum of first two elements of $\mathbf{S}$ are much larger sum of all others, $s_1 + s_2 \gg s_3 + \dots + s_N$, then we can approximate the SVD as,

$$\mathbf{X} \approx (\mathbf{U}_1 \ \mathbf{U}_2) \begin{pmatrix} s_1 & 0 \\ 0 & s_2 \end{pmatrix} (\mathbf{V}_1 \ \mathbf{V}_2)^T,$$

For the i -th sample, this becomes,

$$\mathbf{x}_i \approx (\mathbf{U}_1 \ \mathbf{U}_2) \begin{pmatrix} s_1 & 0 \\ 0 & s_2 \end{pmatrix} \begin{pmatrix} v_{i,1} \\ v_{i,2} \end{pmatrix} = (\mathbf{U}_1 \ \mathbf{U}_2) \begin{pmatrix} s_1 v_{i,1} \\ s_2 v_{i,2} \end{pmatrix},$$

we can then define a transformed 2-element set of coordinates—note these are simply the 1st and 2nd principle component scores,

$$\hat{\mathbf{X}}_i = \begin{pmatrix} s_1 v_{i,1} \\ s_2 v_{i,2} \end{pmatrix}$$

If we take the difference between two points $\mathbf{X}_i$ and $\mathbf{X}_j$, we can write this in terms of the transformed coordinates,

$$\mathbf{X}_i - \mathbf{X}_j \approx (\mathbf{U}_1 \ \mathbf{U}_2)(\hat{\mathbf{X}}_i - \hat{\mathbf{X}}_j)$$

If we look at the squared deviations,

$$(\mathbf{X}_i - \mathbf{X}_j)^T(\mathbf{X}_i - \mathbf{X}_j) \approx (\hat{\mathbf{X}}_i - \hat{\mathbf{X}}_j)^T(\mathbf{U}_1 \ \mathbf{U}_2)^T(\mathbf{U}_1 \ \mathbf{U}_2)(\hat{\mathbf{X}}_i - \hat{\mathbf{X}}_j)$$

which is simply,

$$(\mathbf{X}_i - \mathbf{X}_j)^T(\mathbf{X}_i - \mathbf{X}_j) \approx (\hat{\mathbf{X}}_i - \hat{\mathbf{X}}_j)^T(\hat{\mathbf{X}}_i - \hat{\mathbf{X}}_j),$$

or more simply,

$$d^2(\mathbf{X}_i, \mathbf{X}_j) \approx \hat{d}_{ij}^2 = (\hat{X}_{1,i} - \hat{X}_{1,j})^2 + (\hat{X}_{2,i} - \hat{X}_{2,j})^2,$$

where $d(\mathbf{X}_i, \mathbf{X}_j)$ is the true multivariate distance and $\hat{d}_{ij}$ is the MDS approximated distance. Of course you could include additional dimension to compute the distance. For instance, for three dimensions, you would obtain,

$$d^2(\mathbf{X}_i, \mathbf{X}_j) \approx \hat{d}_{ij}^2 = (\hat{X}_{1,i} - \hat{X}_{1,j})^2 + (\hat{X}_{2,i} - \hat{X}_{2,j})^2 + (\hat{X}_{3,i} - \hat{X}_{3,j})^2$$

```
% let's use the average chemistry that we computed for common rock types in
% the radar plot section
for i = 1:length(R.types)
    for j = 1:length(R.types)
        D(i,j) = sqrt(sum((R.vals(i,:) - R.vals(j,:)).^2));
    end
end

% find a configuration with those inter-point distances
[Y eigenvals] = cmdscale(D);

Dtriu = D(find(tril(ones(length(R.types)),-1)))';
maxrelerr = max(abs(Dtriu-pdist(Y(:,1:2))))./max(Dtriu)
```

```
maxrelerr =
    0.002818517777406
```

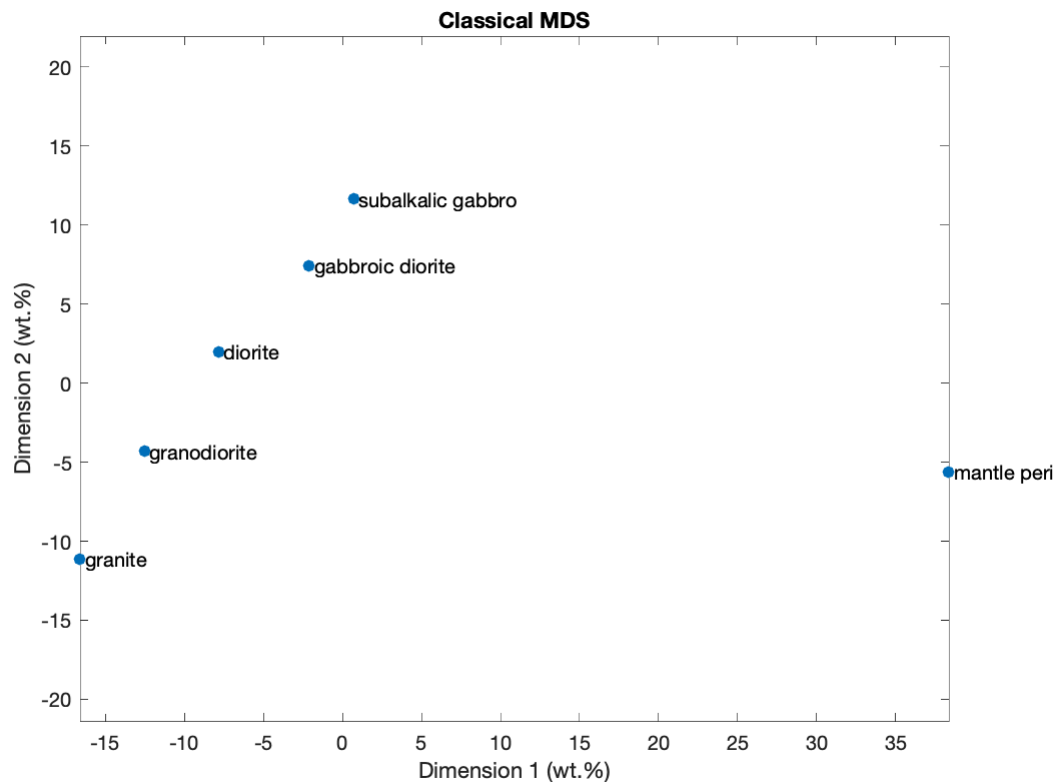
```
figure;
scatter(Y(:,1),Y(:,2),'o','filled')
text(Y(:,1)+0.3,Y(:,2),R.types)

xlabel('Dimension 1 (wt.%)');
ylabel('Dimension 2 (wt.%)');
```

```

title('Classical MDS');
set(gca, 'Box', 'on');
axis equal;

```



The relative distances between points that result from classical MDS gives an estimate of the similarity (close) to dissimilarity (far) from each other on the plot.

8.5.2 Non-classical MDS

In the classical model, we approximate the observed distance by a dimensionally reduced transformed data,

$$\hat{d}_{ij} \approx d_{ij} = ||\mathbf{X}_i - \mathbf{X}_j||_2.$$

However, in the non-classical MDS, we wish to improve our distance estimate and incorporate qualitative (ordinal) data as well. If all the data included in the analysis are quantitative, we perform metric MDS whereas if some or all of the data are qualitative, we perform a non-parametric MDS analysis. We will change our distance approximation that we wish to minimize,

$$f(\hat{d}_{ij}) \approx d_{ij} = ||\mathbf{X}_i - \mathbf{X}_j||_2$$

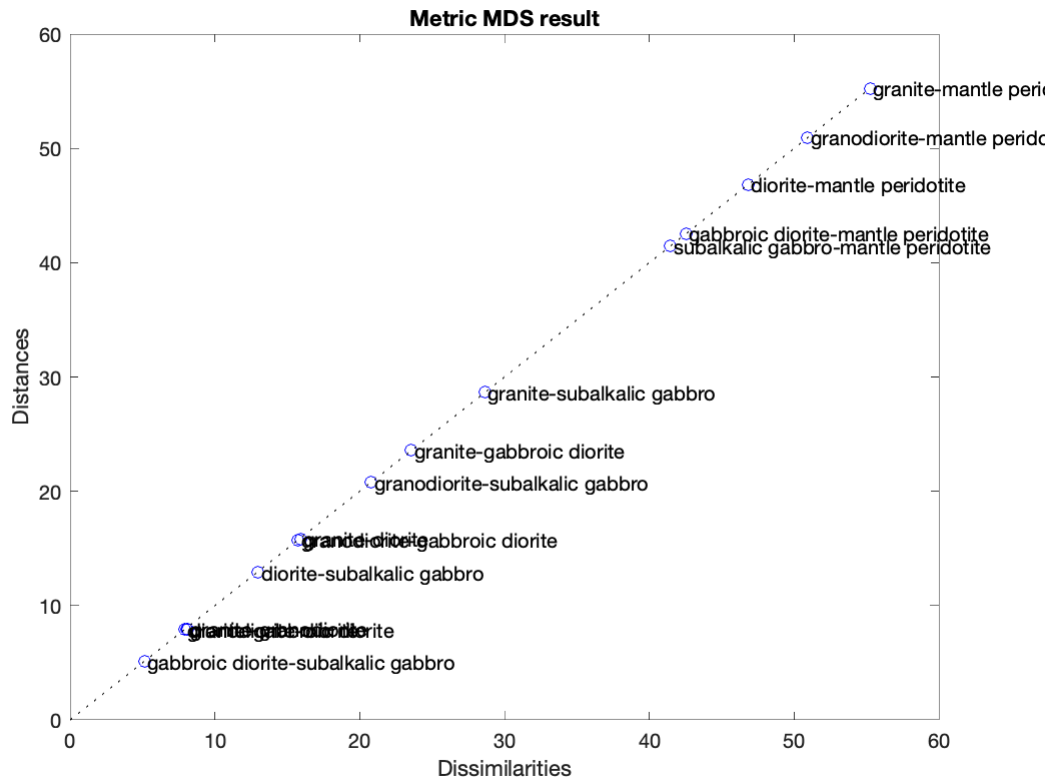
where $f(\hat{d}_{ij})$ is a monotonic function, e.g., $f(\hat{d}_{ij}) = m\hat{d}_{ij} + b$. The function to minimize is called the stress (no relation to pressure) in MDS analysis,

$$\text{stress} = \left(\frac{\sum_{i < j} (d_{ij} - f(\hat{d}_{ij}))^2}{\sum_{i=1}^N \hat{d}_{ij}^2} \right)^{1/2}$$

```
% let's use the average chemistry that we computed for common rock types in
% the radar plot section
dissimilarities = pdist(R.vals);

[Y, stress] = mdscale(dissimilarities, 2, 'criterion', 'metricsstress');
distances = pdist(Y);

figure;
plot(dissimilarities, distances, 'bo', ...
     [0 max(dissimilarities)], [0 max(dissimilarities)], 'k:');
k = 1;
for i = 1:length(R.types)
    for j = i+1:length(R.types)
        text(dissimilarities(k)+0.2, distances(k), [R.types{i}, '-', R.types{j}])
        k = k + 1;
    end
end
xlabel('Dissimilarities');
ylabel('Distances');
title('Metric MDS result')
```



8.6 Linear Discriminant Analysis

PCA and MDS do not require the data to be divided into samples classes (e.g., rock types, site, species, etc.), thus providing a way to produce a score or distance from every point to every other point. Using the average of each class as we did above we can obtain a metric for the differences between classes. However, for unknown samples, it is perhaps not clear to which class the data should belong especially when there is overlap between the classes. Linear discriminant analysis (LDA) allows one to separate samples based on the differences between the classes. So imagine we collect a dataset with M dimensions and N samples. The data can be subset into K unique classes.

The first step to perform a LDA is to determine the mean of each dimension within each class, resulting in a vector $\bar{\mathbf{x}}_i$ and the L_2 distance between each sample within a class and the mean, which is called the scatter,

$$\mathbf{S}_i = (\mathbf{X}_i - \mathbf{1}_{n_i} \bar{\mathbf{x}}_i)^T (\mathbf{X}_i - \mathbf{1}_{n_i} \bar{\mathbf{x}}_i)$$

where $\mathbf{X}_i$ is the subset of $\mathbf{X}$ with class i , n_i is the number of samples in class i , and $\mathbf{1}_{n_i}$ is a column vector of ones with length n_i . $\mathbf{S}_i$ is simply the covariance matrix of class i . The **within class scatter** (pooled covariance matrix) is given by the sum over each class,

$$\mathbf{S}_w = \sum_{i=1}^K \mathbf{S}_i$$

We now need to know how the class averages vary with respect to each other. We compute a quantity called the **between class scatter** (covariance matrix between the classes), which is computed using the mean of all the pooled data, $\bar{\mathbf{x}}$,

$$\mathbf{S}_b = \sum_{i=1}^K n_i (\bar{\mathbf{x}}_i - \bar{\mathbf{x}})^T (\bar{\mathbf{x}}_i - \bar{\mathbf{x}})$$

Now we wish to solve for the eigenvalues and eigenvectors of the system

$$\mathbf{S}_w^{-1} \mathbf{S}_b \mathbf{V} = \Lambda \mathbf{V}$$

just as we would for PCA. If you use the `eig.m` function in MATLAB, you can obtain both the eigenvalues and eigenvector matrix. You can normalize the eigenvalues to 1, $\lambda_i (\sum_{i=1}^K \lambda_i)^{-1}$, to obtain the explained variance for each eigenvector.

```
% MATLAB does not have a linear discriminant analysis function, but can the
% code below could be written to do so.

% columns of data to be used
oxides = {'sio2', 'tio2', 'al2o3', 'feo_tot', 'mgo', 'cao', 'na2o', 'k2o', 'p2o5'};

% rock types analyzed
% use all
% rock_types = unique(data.rock_type);
% use subset
rock_types = {'granite', 'granodiorite', 'diorite', 'gabbroic diorite', 'subalkalic gabbro'}
```

```

% create the vector of ones
one = ones([height(data),1]);

% initialize within and between class scatter matrices
Sw = zeros(length(oxides));
Sb = zeros(length(oxides));

% if only a partial list of rock types are analyzed
aind = zeros([height(data),1]);
for i = 1:length(rock_types)
    aind = aind | strcmp(data.rock_type,rock_types{i});
end

% mean of each column for all the data
datamean = one(aind)'*data{aind,oxides}/sum(aind);

% compute within and between class scatter matrices
for i = 1:length(rock_types);
    % find class i
    ind = strcmp(data.rock_type,rock_types{i});

    % number of samples in class
    ni(i) = sum(ind);

    % mean of columns in class subset of data
    classmean(i,:) = one(ind)'*data{ind,oxides}/ni(i);

    % covariance of class
    Si = (data{ind,oxides} - one(ind)*classmean(i,:))'*(data{ind,oxides} - one(ind)*classmean(i,:));
    % within class scatter matrix (total covariance)
    Sw = Sw + Si;

    % between class scatter matrix
    Sb = Sb + ni(i)*(classmean(i,:) - datamean)'*(classmean(i,:) - datamean);
end

% compute eigenvalues and eigenvectors
[V,D] = eig(inv(Sw)*Sb,'vector');

% compute explained variance
explained = D./sum(D);

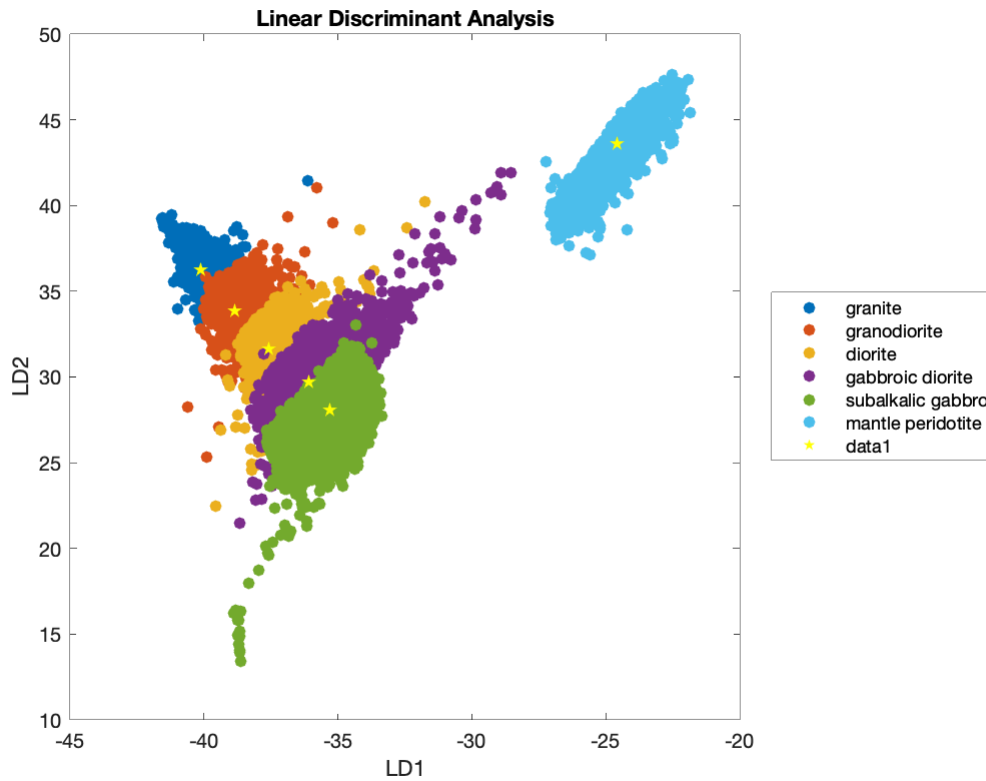
figure;
hold on;
% plot the LD scores
for i = 1:length(rock_types);
    ind = strcmp(data.rock_type,rock_types{i});
    scatter(data{ind,oxides}*V(:,1),data{ind,oxides}*V(:,2),'o','filled');
end
legend(rock_types,'Location','EastOutside');

% plot mean of the classes
scatter(classmean*V(:,1),classmean*V(:,2),60,'yp','filled');

```



```
set(gca, 'Box', 'on');
xlabel('LD1');
ylabel('LD2');
title('Linear Discriminant Analysis');
```



You will notice that the LDA result is significantly different than PCA because PCA finds the maximum similarity between samples whereas LDA tries to find the maximum differences. The result is a better discrimination between classes.